DogCare

Plataforma móvil de servicios para el cuidado de perros

ARQUITECTURA DE SOFTWARE

Arquitectura de Software

Proyecto	DogCare — Plataforma móvil de servicios para el cuidado de perros
Código del documento	DOC-02
Versión	1.0
Fecha	2026-11-02
Estándar de referencia	—
Institución	Universidad de San Buenaventura
Asignatura	Desarrollo de Aplicaciones para Dispositivos Móviles — Optativa II
Docente	Ing. Jairo Armando Salcedo Aranda

Equipo de trabajo

Juan Esteban Olaya Muñoz

Nicolas Quiroz Gomez

Juan Sebastian Garzon Suarez

Tabla de contenido

1. Objetivo y alcance . . . 3

2. Arquitectura general . . . 3

3. Componentes y responsabilidades . . . 4

4. Comunicación entre componentes . . . 5

5. Servicios externos y Firebase . . . 5

6. Seguridad . . . 5

7. Persistencia . . . 6

8. Despliegue . . . 6

9. Decisiones técnicas . . . 6

10. Limitaciones . . . 6

1. Objetivo y alcance

Describe la arquitectura de software de DogCare tal como está implementada en el código: capas, componentes, comunicación, servicios externos, seguridad, persistencia y despliegue. Cubre la aplicación Android y los servicios en la nube que consume; no existe backend propio.

2. Arquitectura general

DogCare es una aplicación Android nativa escrita en Kotlin con Jetpack Compose y Material 3. Se organiza en tres capas dentro de un único módulo `app`: **presentación** (`presentation`, `ui/theme`), **dominio** (`domain`) y **datos** (`data`). No usa ViewModels: el estado de interfaz vive en los Composables (`remember`, `mutableStateOf`) y en los repositorios, que exponen `Flow` y `StateFlow`.

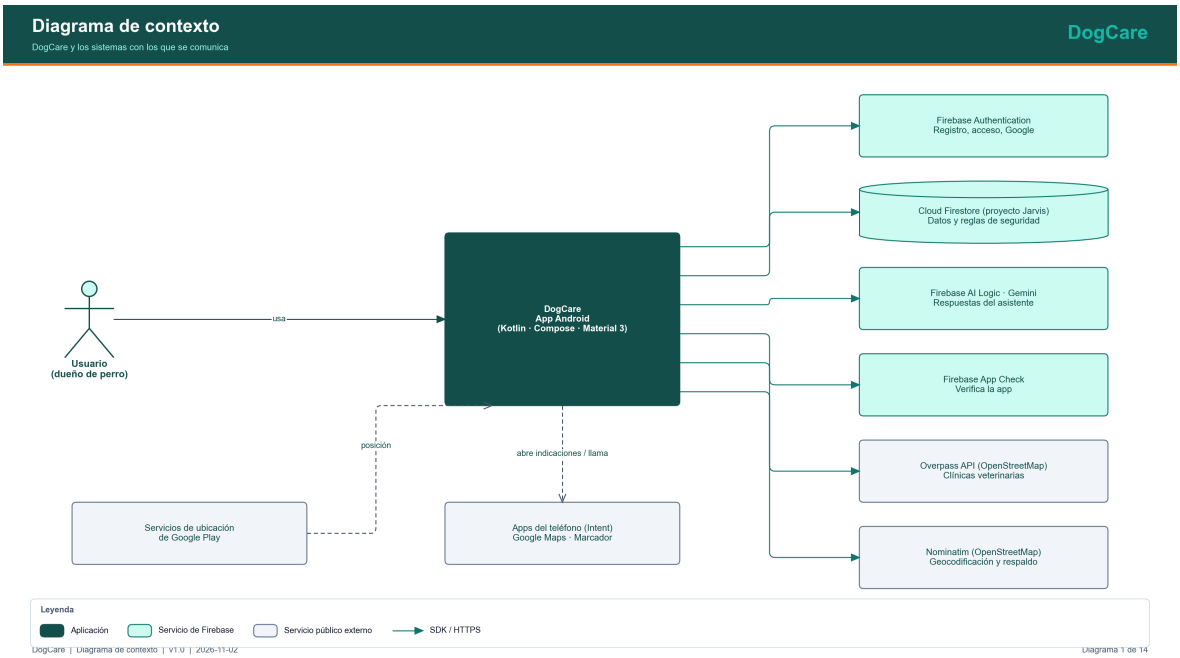


Figura 1. Contexto del sistema.

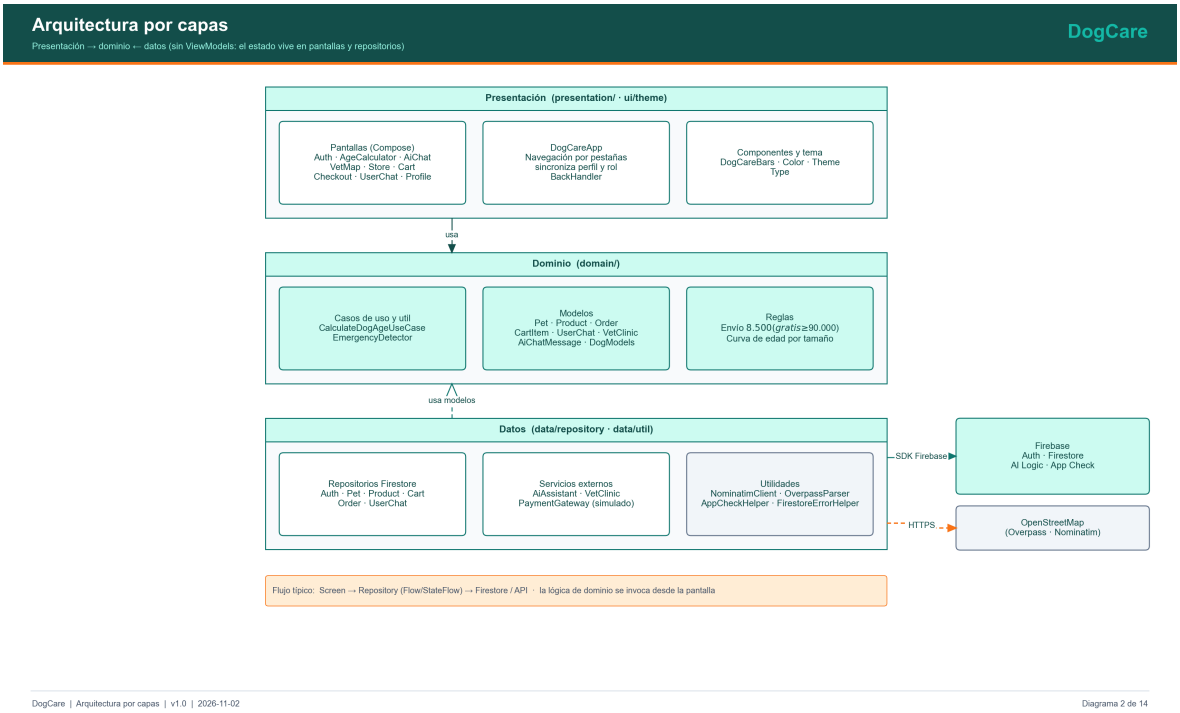


Figura 2. Arquitectura por capas.

3. Componentes y responsabilidades

Capa	Componente	Responsabilidad
Presentación	`DogCareApp`	Navegación por pestañas, sincronización de perfil y rol, `BackHandler`, historial del chat con IA por usuario.
Presentación	`AuthScreen`, `AgeCalculatorScreen`, `AiChatScreen`, `VetMapScreen`, `StoreScreen`, `CartScreen`, `CheckoutScreen`, `UserChatScreen`, `ProfileScreen`	Una pantalla por módulo funcional.
Presentación	`DogCareBars`, `ui/theme`	Barras superior e inferior; paleta verde esmeralda y naranja.
Dominio	`CalculateDogAgeUseCase`	Curva no lineal de edad humana equivalente, etapa de vida y consejos.
Dominio	`EmergencyDetector`	Detección de señales de urgencia en el texto del usuario.
Dominio	`domain/model/*`	Pet, Product, Order, CartItem, UserProfile, Conversation, ChatMessage, VetClinic, AiChatMessage.
Datos	`AuthRepository`	Firebase Authentication (correo y Google), perfil y rol en Firestore.
Datos	`PetRepository`, `ProductRepository`, `CartRepository`, `OrderRepository`, `UserChatRepository`	Acceso a Firestore con `Flow`; el carrito es un `StateFlow` en memoria con respaldo en Firestore.
Datos	`AiAssistantRepository`	Firebase AI Logic (Gemini) con App Check, lista de modelos candidatos y límites de tiempo.
Datos	`VetClinicRepository`, `NominatimClient`, `OverpassParser`	Veterinarias reales de OpenStreetMap; distancia Haversine; respaldo Nominatim.
Datos	`PaymentGatewayRepository`	Pago SIMULADO local con estados aprobado, rechazado y pendiente.
Datos	`AppCheckHelper`, `FirestoreErrorHelper`	Configuración de App Check y manejo uniforme de errores de Firestore.

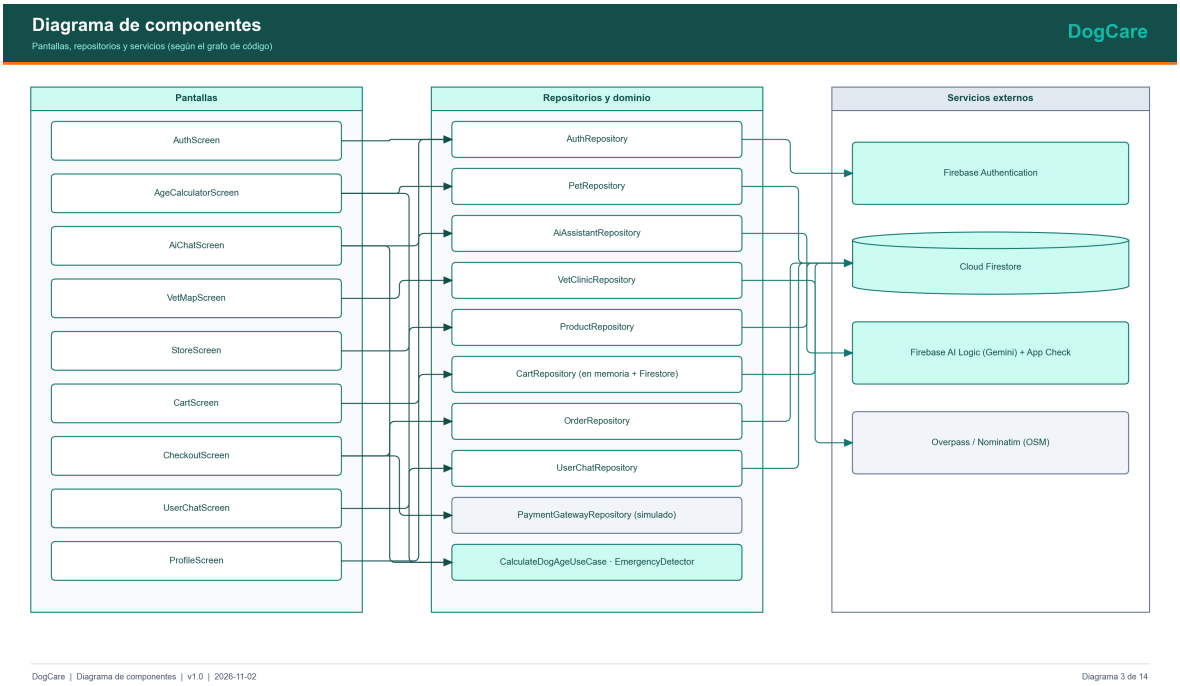


Figura 3. Componentes: pantallas, repositorios y servicios.

4. Comunicación entre componentes

- Las pantallas reciben los repositorios por parámetro desde `MainActivity` (inyección manual, sin framework).
- Los repositorios exponen `Flow` de Firestore; ante un error se registra y se emite una lista vacía en lugar de lanzar la excepción.
- Las llamadas de red usan corrutinas (`Dispatchers.IO`) con tiempos máximos explícitos.
- La IA y Firestore se acceden mediante SDK; Overpass y Nominatim mediante HTTPS con OkHttp y un `User-Agent` propio.

5. Servicios externos y Firebase

Servicio	Uso	Observación
Firebase Authentication	Registro, acceso, Google, recuperación de contraseña	Proyecto Firebase `Jarvis`.
Cloud Firestore	Datos de la aplicación	Base dedicada; reglas en `firestore.rules`.
Firebase AI Logic (Gemini)	Asistente de IA	Modelos en `MODEL_CANDIDATES`: `gemini-3.5-flash-lite`, `gemini-3.8-flash`.
Firebase App Check	Verificar la app	Proveedor de depuración en desarrollo; reCAPTCHA declarado.
Overpass / Nominatim	Veterinarias y geocodificación	Servicios públicos sin clave; límites de uso y de tiempo.

6. Seguridad

- Reglas de Firestore con denegación por defecto, validación de esquema, rol de solo lectura para el cliente y acceso limitado al dueño o a los participantes.
- App Check para evitar uso de la API de IA desde apps no autorizadas.
- La clave de API de Android de Firebase no es un secreto de servidor, pero debe restringirse por paquete y SHA-1; `google-services.json` no se publica en el repositorio.
- El pago es simulado: no se procesan ni almacenan datos de tarjeta.

7. Persistencia

Toda la persistencia es remota en Cloud Firestore. No existe base local: la dependencia Room estaba declarada sin ningún uso y se retiró. El carrito se conserva en memoria y se sincroniza con `carts/{uid}`.

8. Despliegue

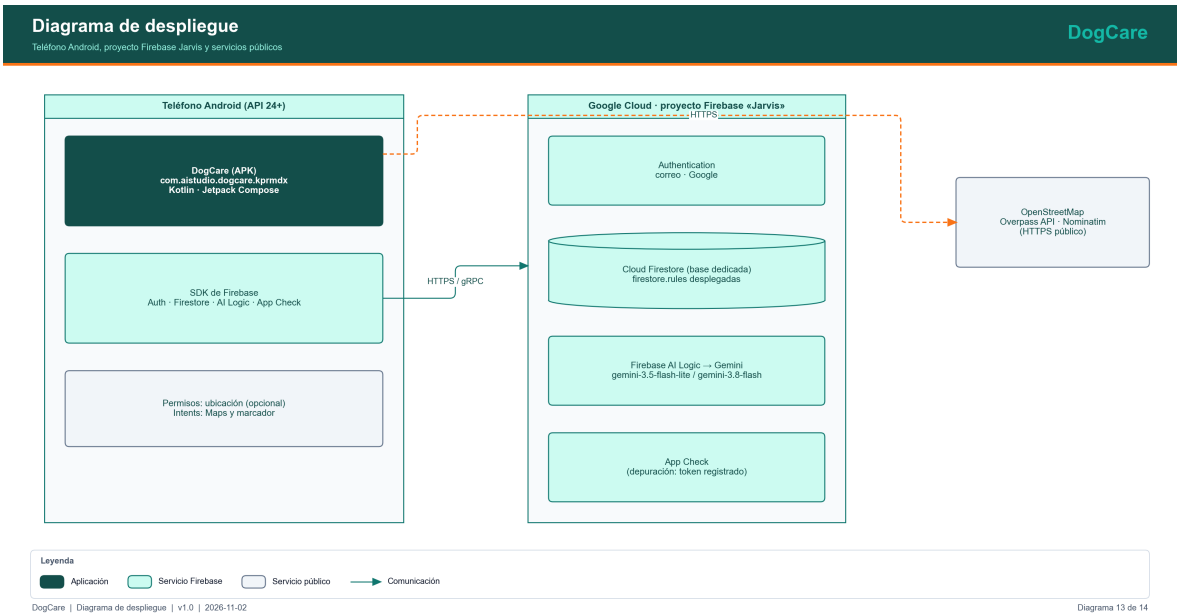


Figura 4. Despliegue.

9. Decisiones técnicas

Decisión	Motivo	Consecuencia
Firebase en lugar de backend propio	Sin costo de servidor, autenticación y reglas integradas.	No hay validación de servidor para precios ni pagos.
OpenStreetMap en lugar de Google Maps	Directorio verificable sin clave ni costo.	Teléfonos y marca de urgencia dependen de las etiquetas OSM.
Pago simulado	Decisión del autor: sin pasarela real ni cobros.	El cliente decide el estado del pago (RK-01).
Marketplace multivendedor	Cualquier usuario autenticado publica sus productos.	Reglas por `sellerUid` en lugar de un rol de tienda.
Sin ViewModels	Aplicación de alcance académico; estado simple.	Más lógica en Composables; menos testeable.

10. Limitaciones

- Sin backend: no hay verificación de pagos ni precios en servidor.
- Dependencia de servicios de terceros sin SLA.
- Arquitectura sin ViewModels: se recomienda migrar si el proyecto crece.